PREPARED BY LITHIKA DHARMASENA

OCTOBER 2026

MMA3001

FORECAST-DRIVEN SOLAR BATTERY CONTROL

INDIVIDUAL PROJECT REPORT

34976639

LDHA0002@STUDENT.MONASH.EDU

SUMMARY

This project asks whether a household with rooftop solar and a home battery can lower its electricity bill by planning battery use around forecast wholesale prices, and how much of the achievable saving is lost because forecasts are imperfect. Stage 1 forecasts the hourly Victorian spot price 72 hours ahead from calendar, weather and demand information. Stage 2 uses the forecast in a linear-programming controller for a 6.6 kW solar array and a 10 kWh, 5 kW battery, simulated hour by hour over 359 days (1 October 2025 to 24 September 2026).

A support vector regression (SVR) model gave a price error of $31.72/MWh (mean absolute error, MAE), compared with $53.78/MWh for the baseline of repeating the price from one week earlier. The forecast-driven controller reduced the simulated annual bill from $1,179.84 (no battery) to $590.75, against $643.46 for a simple rule and $546.13 for a controller with perfect price knowledge. Forecast error therefore cost $44.62 a year, and the forecast-driven controller beat the simple rule only while forecast MAE stayed below about $43/MWh. The explicit Euler solution of the battery equation was verified against the analytical solution, and the forecasts were validated by two independent methods. An interactive advisor built on the same code is hosted online.

Repository: github.com/lithdhar/MMA3001-Project. Online advisor: mma3001-battery-advisor.streamlit.app.

1. ENGINEERING PROBLEM

1.1 CONTEXT AND PROBLEM

Victorian households with rooftop solar are increasingly installing batteries. A battery is an energy-storage device with finite capacity, a power limit, conversion losses and a slow self-discharge, so its stored energy behaves as a first-order dynamic system driven by a control decision. Under a spot-linked tariff the energy price changes every hour. Over the study period 3,638 hours (about 23%) had a negative average price and the price peaked at $19,070/MWh. A battery that charges when energy is cheap or surplus and discharges when it is dear can therefore reduce a bill substantially, provided the future price is known. It is not known, so the controller must work from a forecast.

The engineering problem is to decide, for each hour, whether the battery should charge, discharge or stay idle so that the household bill is minimised, and to quantify how much that decision depends on forecast accuracy. The question is of practical importance because the same forecast-error question applies to any energy-storage scheduling problem.

1.2 INTENDED USE AND SCOPE

The computational solution is intended as a decision-support and evaluation tool. It compares four battery controllers on a common simulated household and reports the value of price forecasting. It is not a control system for a real battery. The scope was limited deliberately: one location (Melbourne weather, Victorian prices), one tariff structure, one typical household load profile, a fixed solar and battery specification, and one-hour time steps.

2. INPUTS, OUTPUTS AND DATA

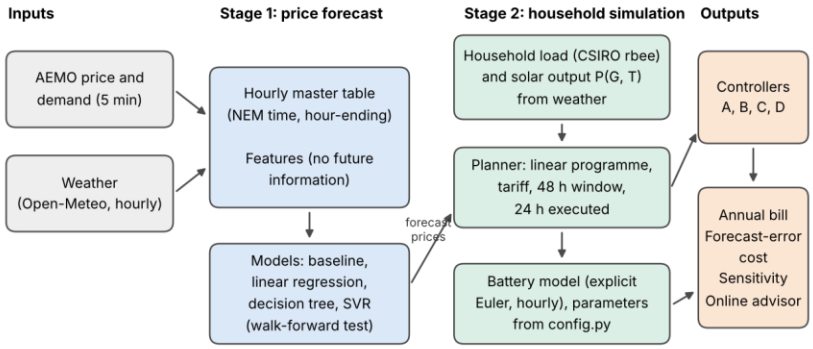


Figure 1. Workflow of the computational solution. Stage 1 produces forecast prices; Stage 2 turns them into battery decisions and a bill.

Table 1. Input data and parameters accepted by the solution.

Input	Format, units and domain	Source	Use
Spot price and demand	– $/MWh – Up to 19,070 observed – Negative values occur – MW 1 Dec 2024 to 24 Sep 2026 – Averaged from 5-minute values	AEMO aggregated price and demand, VIC1	– Forecast target and demand feature – Bill calculation
Weather	– Hourly – Temperature (°C) – Wind at 10 m and 100 m (m/s) Global, direct and diffuse radiation (W/m^2)	Open-Meteo historical forecast API, Melbourne	– Forecast features – Solar output
Household load	– kWh (Victoria) – Half-hourly – Typical use by month – Weekday or weekend	CSIRO rbee extract (supplied by M. Ambrose)	– Household demand – Scaled for household size
Tariff	– Network and other charges 15 c/kWh – GST 10% – Supply charge $1.2114/day – Feed-in at spot price (floored at zero)	Assumed spot-linked retail plan (CitiPower area)	– Bill and planning prices
System parameters	– 6.6 kW array – Performance ratio 0.8 – Battery 10 kWh 5 kW, – 90% round trip – Self-discharge 0.001 h−1	src/config.py	– Solar and battery models

All series were aligned to one clock, National Electricity Market (NEM) time (UTC+10 with no daylight saving), with each timestamp marking the end of its hour. Alignment was checked, the weather solar peak stayed between about 12:36 and 13:06 hours through the year, and the Victorian load profile correlated with AEMO demand at 0.946 at zero lag and less at other lags. These checks are automated in a test. The merged master table has 15,911 hourly rows. The evaluation year contains 8,616 hours; negative prices and price spikes were retained because they are real events that the controller must handle.

Outputs. Stage 1 produces a forecast price in $/MWh for every hour. Stage 2 produces the hourly battery power (kW, positive when charging) for each controller, stored energy (kWh), grid import and export (kWh) and the annual bill ($), together with the forecast-error cost, defined as the bill of the forecast-driven controller minus that of the perfect-foresight controller. **Invalid input** is handled explicitly, the master table is tested for gaps, duplicate hours and daylight-saving jumps; the planner raises an error if the optimisation fails; and the online advisor refuses a day whose

highest temperature is below its lowest, or a month and sky combination for which no typical day exists.

33. COMPUTATIONAL APPROACH

The solution is a two-stage workflow (Figure 1) written in Python. Stage 1 forecasts the hourly price 72 hours ahead. Stage 2 simulates a household with solar and a battery and compares controllers, one of which plans with the forecast. All settings are held in one file (*src/config.py*).

3.1 STAGE 1: PRICE FORECAST

The forecast horizon was set to 72 hours, so that the controller can plan with a forecast made three days earlier. The 18 model inputs are calendar, weather and demand for the target hour, and price information at least as old as the horizon: the price 72 hours earlier, one week earlier and two weeks earlier, and 24-hour and 7-day rolling means ending 72 hours before the target. This rule is the central design constraint of Stage 1, since a forecast may only use information available when it is made. It is enforced in code and tested.

Four forecasts were compared. The **baseline** repeats the price from the same hour one week earlier; it has no fitted parameters and represents what a household could do without a model. **Linear regression** fits a weighted sum of the inputs by least squares and is the interpretable benchmark. A **decision tree** splits the hours into groups with similar inputs and predicts the average price of each group, which captures non-linear effects and interactions. **Support vector regression (SVR)** with a radial basis function kernel fits a smooth non-linear function, ignores errors below a tolerance ε and penalises larger errors in proportion to their size (with weight C), so one price spike has less influence than under least squares. It was chosen as the model expected to cope best with a noisy, spiky series, and Section 5 tests this.

Settings. Decision tree: maximum depth 8, at least 100 hours per leaf, fixed random seed. SVR: C = 0.1, ε = 0.1 (in standardised price units) and the default kernel width of scikit-learn, with inputs and target standardised. Scalers were fitted on training data only. These values were chosen by grid search on the last 60 days of the training window, never on the test year (trial MAE \$32.8/MWh at C = 0.1 and \$40.1/MWh at C = 3).

3.2 STAGE 2: PHYSICAL MODELS

Solar output follows a standard temperature-corrected model, with G the radiation in W/m^2, T the air temperature in °C, PR the performance ratio, γ the temperature coefficient (0.004 per °C) and h the cell-heating factor (0.03 °C per W/m^2):

$$P_{pv} = P_{rated}\,(G / 1000)\,PR\,[1 - \gamma\,(T + hG - 25)] \quad (1)$$

The battery energy E (kWh) obeys a linear first-order equation, with charge power P_{ch}, discharge power P_{dis}, one-way efficiencies $\eta_c = \eta_d = \sqrt{0.90}$ and self-discharge rate k:

$$dE/dt = \eta_c\,P_{ch} - P_{dis} / \eta_d - kE \quad (2)$$

Equation (2) was integrated with the explicit Euler method at a one-hour step. In each hour the self-discharge loss is removed first, and the requested power is then cut back so that the power limit (5 kW) and the energy limits (0 to 10 kWh) are respected. Explicit Euler was selected because the equation is linear, the hourly step is 2,000 times smaller than the stability limit and the discretisation error is a small fraction of the capacity (Section 4). Every hour the household balance holds, which fixes the grid flows (import I, export X, kWh):

$$P_{pv} + P_{dis} + I = L + P_{ch} + X \quad (3)$$

The buy price is (spot + 15 c/kWh) × 1.10 and the feed-in price is the spot price, floored at zero. A fixed supply charge of $1.2114 a day (a total of $434.89 over the study period, 37% of the no-battery bill) is included in every bill.

3.3 CONTROLLERS AND PLANNER

Four controllers are compared on identical solar and load series. **A** has no battery. **B** is the simple rule: surplus solar charges the battery, a shortfall is met from it, and the battery never charges from the grid. **C** plans with the SVR forecast and **D** plans with the actual prices; D is the best the planner can do and bounds what any forecast could achieve. A further controller, **C2**, plans with the baseline last-week forecast. C, C2 and D use the same planner, which at the start of each day is given the next 48 hours of solar output, load and prices. For each hour it chooses charge power, discharge power, grid import, grid export and stored energy (5 × 48 = 240 unknowns) to minimise the cost of the energy bought and sold:

$$\min \ \Sigma_t \left(\pi_{\text{buy},t} \, I_t - \pi_{\text{fit},t} \, X_t \right) \tag{4}$$

Here $\pi_{\text{buy},t}$ and $\pi_{\text{fit},t}$ are the planning buy and feed-in prices and the sum is over the 48 hours of the window. The constraints are the battery equation (2) for each hour, the balance (3), $0 \le E \le 10$ kWh, $0 \le P_{\text{ch}}, P_{\text{dis}} \le 5$ kW and $I, X \ge 0$. All are linear, so this is a linear programme, solved exactly by the HiGHS solver through SciPy with no tuning parameters. The planner is re-run every 24 hours from the battery's real energy and only the first 24 hours of the plan are carried out, so the limits apply to what actually happens. All controllers are billed at actual prices.

A linear programme was chosen over a rule-based threshold, which needs tuned price levels and cannot anticipate a spike, and over dynamic programming, which needs the stored energy divided into levels and is only approximate. It is exact for this model and solves in milliseconds. The 48-hour window was used because no value is placed on energy left in the battery at the end of a window: with perfect foresight a 24-hour window gave a bill of $667.24 against $546.13 for 48 hours (Table 5). The planning buy price is never allowed below the feed-in price; without this floor a negative buy price lets the planner profit from importing and exporting the same energy, which no real meter allows.

3.4 ASSUMPTIONS, IMPLEMENTATION AND LIMITATIONS

Assumptions. Weather and demand for the target hour are actual values standing in for forecasts. The planner knows solar output and load exactly, so the only difference between C and D is the price forecast. The battery has constant efficiency, self-discharge proportional to the stored energy and no degradation or cycling cost, and every simulation starts half full. Household load is a typical profile rather than a measured one, the household is a price taker, and one assumed tariff applies.

Implementation. The workflow uses Python 3.10 with pandas and NumPy for the data, scikit-learn for the models and scalers, SciPy for the planner (*linprog* with HiGHS) and the reference solvers of Section 4, and Matplotlib for figures. The code is divided into modules in *src/*, one for each task, and notebooks run the analysis in numerical order by calling them. Package versions are fixed in *requirements.txt* and the only random element, the decision tree, has a fixed seed, so re-running the notebooks gives the same results. Tests in *tests/* check each module (Section 5.3).

Known limitations. The SVR gives a point forecast without uncertainty and over-predicts on average (bias +$15.65/MWh, Table 3). Actual weather and demand make its error optimistic, and the planner's exact knowledge of solar and load favours every planning controller. Explicit Euler is first-order and stable only for $\Delta t < 2/k$, so a faster battery model would need a smaller step or an implicit method. The planner ignores everything beyond 48 hours, and the battery model omits degradation and the change of efficiency with power. Sections 5.4 and 7 discuss their effect.

4. ALTERNATIVE SOLUTIONS

Three decisions in the solution had credible alternatives: the price forecast, the battery controller and the time integration of the battery equation. In each case the alternatives were run on the same data and compared on the criteria that matter for the application, so that the final design rests on evidence rather than preference. Alternatives that were considered but not run are identified as such.

4.1 PRICE FORECAST

The four forecasts range from a naive benchmark to a flexible non-linear model: the baseline, linear regression, a decision tree and an SVR (Section 3.1). Accuracy under walk-forward and fixed-split testing is given in Table 3; Table 2 adds the other criteria in the brief. Neural networks and tree ensembles were not tested, because the aim is a fair comparison of a few credible alternatives and not a survey of methods.

Table 2. Comparison of the forecast alternatives (359 days, 72 h ahead, walk-forward testing). Best value in bold.

Criterion	Baseline	Linear regression	Decision tree	SVR
MAE ($/MWh)	53.78	45.04	43.31	**31.72**
Monthly MAE, lowest to highest ($/MWh)	39.5 to 84.4	34.0 to 79.5	23.5 to 126.4	**21.3 to 53.8**
Months more accurate than the baseline (of 12)	–	11	10	**12**
MAE in the 15 hours above $300/MWh ($/MWh)	692.7	**459.5**	657.4	509.5
Interpretability	Trivial	High: one coefficient per input	High: splits can be read	Low: no direct read-out
Implementation complexity	None	Low	Low	Moderate: scaling and three settings
Fitting cost	None	Negligible	Low	Highest

The SVR was the most accurate and the most consistent. It was the only model to beat the baseline in every month, and its worst month ($53.8/MWh) was better than the worst month of every other model. The decision tree is accurate in typical months (its best month is $23.5/MWh) but its worst month is $126.4/MWh, which is why its RMSE in Table 3 is the largest: its predictions jump between leaf averages and miss spikes by large amounts. Linear regression is stable but cannot follow the non-linear pattern of prices. No model forecasts spikes: in the 15 hours with a price above $300/MWh the MAE was between $459 and $693/MWh for every model, and linear regression was the best of the four. The advantage of the SVR is therefore in ordinary hours.

The costs of the SVR are lower interpretability and the highest fitting cost, which grows faster than linearly with the number of training rows. For this application the cost is small: fitting all models over all 13 walk-forward folds took 43 s, and a forecast is needed once a day. Memory use was not measured and was not a limiting factor at about 15,000 training rows.

4.2 BATTERY CONTROLLER

The simple rule B needs no forecast and is the baseline for the planner. The planner was run with three price inputs: the baseline last-week forecast (C2), the SVR forecast (C) and the actual prices (D), which gives the upper limit of what any forecast could achieve. Only the baseline and SVR forecasts were passed to the planner. Table 4 gives the bills.

Rule B is the simplest to implement and has no data requirement, but it never charges from the grid and so cannot use cheap or negative-price hours. The planner needs a forecast, weather inputs

and a linear-programme solver; its saving over B and how that saving depends on forecast accuracy are given in Section 5.2, and all five controllers ran in 6.2 s over 359 days. A price-threshold rule and dynamic programming were considered but not implemented (Section 3.3).

4.3 TIME INTEGRATION OF THE BATTERY EQUATION

Five methods were compared for Equation (2): explicit Euler (selected), implicit Euler and the stiff solvers BDF, Radau and LSODA from SciPy. Each was checked against the analytical solution, which exists because the power is constant within each hour. Both Euler methods are first-order: reducing the step from 60 minutes to 1 minute reduced the error from 5.4×10^{-4} kWh to 8.9×10^{-6} kWh, with a fitted order of 1.00 (Figure 2). At the hourly step the error is about 0.005% of the battery capacity, which is small beside the fraction of a kilowatt-hour that matters for the bill. The stiff solvers reduced the error to between 10^{-13} and 10^{-9} kWh, but were 300 to 4,600 times slower (2.6 to 36.8 s against 0.008 s per simulated year) and made no difference to the decisions or the bill.

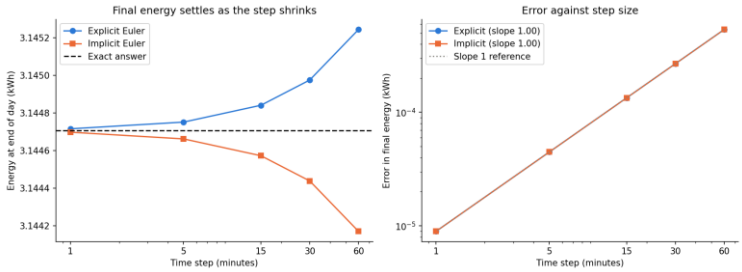


Figure 2. Convergence of explicit and implicit Euler to the analytical end-of-day energy. Both have slope 1.00 on the log-log axes.

Stability does not restrict the choice. For Equation (2) one explicit Euler step multiplies an error by $g = 1 - k\Delta t$, so the method is stable only for $\Delta t < 2/k$, which is 2,000 hours for $k = 0.001$ h^{-1}. The hourly step gives $g = 0.999$, and a numerical check (notebook 09) showed a 0.1 kWh error shrinking to 0.049 kWh after 30 days, as predicted. Implicit Euler, which is stable for any step, therefore offers no benefit here, although it would be needed if the self-discharge rate were large: with $k = 2.5$ h^{-1} the explicit factor is −1.5 and the solution oscillates and grows.

4.4 SELECTION

The selected design is the SVR forecast, the linear-programme planner and hourly explicit Euler. Each was the most accurate option, or the cheapest option accurate enough for its purpose, for one year at one location, and the forecast ranking held under both validation methods (Section 5.1).

5. VALIDATION METHOD AND RESULTS

Four things were validated, each by a method suited to it: the forecast (on withheld data, by two schemes), the decision value (by comparison with bounding controllers and a sweep of forecast error), the numerical and physical consistency of the simulation (against an analytical solution and the energy balance) and the software (by automated tests). The limits of this evidence are stated in Section 5.4.

Acceptance criteria. Results were accepted against stated criteria and not by the appearance of a plot. The forecast had to beat the baseline on MAE and RMSE under both validation schemes and, for control, be accurate enough that planning with it costs less than the simple rule B (a break-even MAE of about $43/MWh, Section 5.2). The simulation had to close the energy balance to zero error, keep the battery between 0 and 100% of capacity and use a time integration that is stable and first-order convergent (Section 4.3). All software tests had to pass.

5.1 FORECAST VALIDATION

The forecast was validated on withheld data with the order of time respected. Random cross-validation was not used, because shuffling would train the model on the future and put neighbouring, strongly correlated hours in both the training and test sets (Bergmeir and Benítez, 2012; Hyndman and Athanasopoulos, 2021). Two schemes forecast the same evaluation year (359 days). In a **fixed split**, each model is fitted once on the 287 days of data before the year and then forecasts the whole year without being updated; this is the stricter test, since the model ages as the year proceeds. In **walk-forward testing**, the year is divided into 28-day blocks and the models are refitted before each block on all earlier data. In both schemes the training data end 72 hours before the forecast period begins, because the latest 72 hours of prices would not yet be known (López de Prado, 2018). Walk-forward testing imitates regular refitting in practice and was the main method; the fixed split shows how much of the result depends on that refitting. Metrics were MAE, which has the same units as price and is robust to spikes; root mean square error (RMSE), which penalises large misses; and bias, to expose systematic over-prediction. The two schemes agree on the ranking (Table 3, Figure 3).

Table 3. Forecast accuracy on 359 days, 72 h ahead ($/MWh). Lowest error in bold.

Model	MAE walk-fwd	RMSE walk-fwd	Bias walk-fwd	MAE fixed	RMSE fixed	Bias fixed
Baseline (price one week earlier)	53.78	91.00	0.07	53.78	91.00	0.07
Linear regression	45.04	69.43	18.51	53.68	78.19	23.53
Decision tree	43.31	115.69	27.68	50.97	125.02	37.80
SVR	**31.72**	**56.09**	15.65	**38.18**	**61.03**	23.87

The SVR had the lowest MAE and RMSE under both schemes and was 41% more accurate than the baseline on the walk-forward MAE. The decision tree had the worst RMSE because it misses spikes by large amounts, and all three fitted models over-predicted on average (positive bias). The fixed split raised the SVR error from 31.72 to 38.18 $/MWh but increased the household bill by only $0.98 ($590.75 to $591.73), so the downstream conclusion does not depend on the validation scheme. A third check with calendar and weather inputs only gave an SVR MAE of 32.85 (walk-forward) and 37.77 $/MWh (fixed split), so most of the skill comes from weather and time of day and not from lagged prices.

Influence on model selection. The tree depth, leaf size and SVR penalty C were chosen by grid search on the last 60 days of the training window, never on the evaluation year, and kept fixed in every fold. The evaluation year was then used to compare the four forecasts, and the SVR was adopted because it had the lowest error under both schemes and in every month.

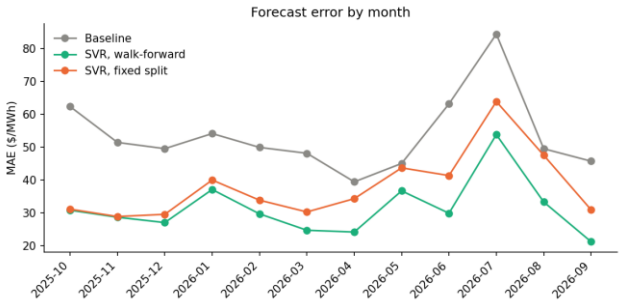


Figure 3. Monthly forecast MAE for the baseline and for the SVR under walk-forward and fixed-split validation.

5.2 DECISION VALIDATION AND CONTROLLER COMPARISON

Table 4. Simulated annual bill over 359 days, including the $434.89 supply charge.

Controller	Bill ($)	Saving vs A ($)	Share of D's saving (%)
A. No battery	1,179.84	0.00	0
B. Simple rule	643.46	536.38	84.6
C2. Planner with last-week forecast	639.02	540.81	85.3
C. Planner with SVR forecast	**590.75**	589.08	93.0
D. Planner with perfect price knowledge	546.13	633.70	100.0

A battery halves the bill even under the simple rule. The planner adds a further $52.71 over B with the SVR forecast, which is 54% of the $97.33 that perfect foresight would add. Forecast error therefore costs $44.62 a year. The baseline forecast is almost worthless for control: C2 beats B by only $4.44. To establish how accurate a forecast must be, the SVR errors were scaled by a multiplier from 0 to 2 and the planner re-run (Figure 4). The forecast-driven controller outperforms the simple rule only while MAE stays below about $43/MWh, which is about 36% above the SVR's error. Beyond this the planner acts on wrong prices and does worse than a rule that ignores prices. Robustness was tested by repeating the comparison for 14 cases that varied battery size, power, efficiency, tariff and forecast horizon (Appendix A): the SVR planner beat B in every case, by $15.10 to $159.68. This is the central result of the project: a forecast is valuable only if it is accurate enough, and the margin is not large.

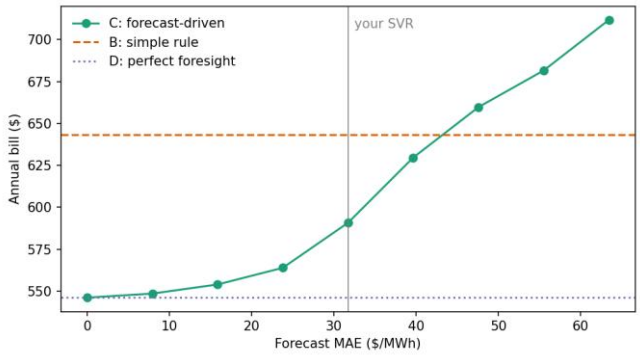


Figure 4. Annual bill of the forecast-driven controller as forecast error is scaled. The crossing with controller B marks the break-even error.

5.3 NUMERICAL, PHYSICAL AND SOFTWARE VERIFICATION

Three checks do not depend on the price data. The time integration was compared with the analytical solution of Equation (2) and with SciPy's solvers (Section 4.3). The energy balance of every controller closed to zero error and the battery state of charge stayed within 0 to 100%, which checks conservation of energy. The repository contains 103 automated tests in 13 files (99% line coverage of *src/*), covering time alignment, the rule that no feature uses future information, metrics and models, the solar, load and battery models against hand calculations, the energy balance and bill, the numerical methods and the controllers. A further test guards the headline results against silent change.

5.4 WHAT THE VALIDATION CANNOT ESTABLISH

No measured data from a real battery or household were available, so the solar model, the load profile and the battery losses are assumptions and the bills are simulated rather than observed. Weather and demand for the target hour are actual values standing in for forecasts, which makes the reported forecast error optimistic. The evaluation year also served to compare the four forecasts and there is no further untouched test set, so the reported error is slightly optimistic for

new data. The test covers one year at one location under one tariff, so seasonal and market conditions outside it, including price regimes with more or fewer spikes, are untested. The validation shows that the method is consistent and robust to the validation scheme, not that the bill in Table 4 would be realised in practice.

6. COMPARISON, OPTIMISATION AND PERFORMANCE

Each major decision was taken from a comparison rather than from preference. The forecast model, controller and time integration are compared in Section 4; Table 5 lists the remaining design decisions, and the sensitivity tests are in Appendix A.

Table 5. Principal design decisions, alternatives and evidence.

Decision	Alternatives	Evidence	Outcome
Forecast model	Baseline, linear regression, decision tree, SVR	MAE $31.72 vs 43.31 to 53.78; best under both validation methods	SVR
Validation scheme	Walk-forward; single fixed split	Ranking and bill agree (bill difference $0.98)	Both reported
Forecast horizon	72, 96, 120 hours	Bill $590.75, $590.26, $590.52	72 h (insensitive)
Planning window	24, 48, 72 hours (perfect foresight)	Bill $667.24, $546.13, $545.77	48 h
Time integration	Explicit Euler, implicit Euler, BDF, Radau, LSODA	Same decisions; solvers 300 to 4,600 times slower	Hourly explicit Euler
Planner	Rule B; linear programme	C saves $52.71 over B	Linear programme

Sensitivity. Battery capacity had the largest influence (Appendix A): the planner's saving over rule B was $15.10 at 5 kWh, $52.71 at 10 kWh and $159.68 at 20 kWh, so the value of forecasting grows with storage. A fixed 5 c/kWh feed-in cut the advantage to $17.57, because exports lose their spot-price upside, and round-trip efficiency from 80% to 95% changed it from $35.59 to $62.63. Battery power had almost no effect.

Performance. Hourly explicit Euler takes about 0.02 ms per simulated day and fewer than 10 floating-point operations per step, so the integration cost is negligible. The cost of the solution is the optimisation: running all five controllers over 359 days took 6.2 s, and the 14 sensitivity cases took 49 s in total.

6.1 ONLINE ADVISOR

As an extension, the same code was packaged as an interactive page in which a user describes a day (month, weekday or weekend, sky condition, temperature range and mean wind) and the solar and battery system, and receives an hourly recommendation and the daily cost under strategies A, B and C. Prices are forecast by an SVR using calendar and weather inputs only, from typical days built from the weather record. This simplification was validated before use: it raised the price MAE from 37.77 to 41.73 $/MWh and the annual bill of C from $601.95 to $608.54, which is still $34.92 below the simple rule.

7. LIMITATIONS AND LESSONS LEARNED

Limitations. The principal limitations are those in Sections 3.4 and 5.4. In addition, battery degradation and the cost of cycling are ignored, which favours aggressive cycling, and the tariff is a single assumed plan.

Most vulnerable part. The part of the solution most exposed to failure is the planner's dependence on the price forecast during volatile periods. Forecast-error cost was concentrated: the five worst days accounted for $28.25 of the $44.62 (63%), July 2026 alone for $15.45, and the median day for

about one cent. A single spike that the forecast misses can therefore cost more than weeks of normal operation. A practical mitigation is a guard that falls back to the simple rule B when forecast uncertainty is high; this was not implemented.

Lessons. First, forecast accuracy and decision value are different quantities. The SVR reduced MAE by 41% relative to the baseline, yet the baseline controller was barely better than the simple rule while the SVR saved a further $48, and the loss from forecast error was concentrated in a handful of volatile days. Second, validation design is part of the result: building the fixed-split alternative showed that the conclusion held but that the error figure was optimistic by about $6/MWh. Third, the actual parameters lie a factor of 2,000 inside the stability limit. Fourth, small modelling conventions have large consequences: the buy-price floor and the hour-ending timestamp convention each removed errors that would otherwise have produced plausible but wrong bills.

Proposed improvements. Replace actual weather and demand by archived forecasts; benchmark the forecast against AEMO's own pre-dispatch (PD7DAY) forecasts; extend the price history beyond about 22 months so that a separate tuning and test period is available; add a fallback to rule B when forecast uncertainty is high; model battery degradation; and validate the household model against metered data.

8. USE OF AI AND REFLECTION

Tools and purpose. Claude (Anthropic) was used throughout the project as an assistant for explanation, planning, debugging and drafting. It explained numerical concepts such as stability and convergence at the level needed to implement them, proposed the order of work as a series of steps, suggested code for each module that I then typed, ran and committed myself, helped diagnose environment and Git errors, and produced the first draft of the notebook commentary and of this report from my results files. The online advisor was my idea; the assistant proposed its structure and the day-description approach.

Level of contribution. AI contribution was high for code drafting and explanation, and for the first draft of this document. Responsibility for the problem definition, the research question, the data choices, the validation design, the interpretation of results and the final text remained mine.

Verification. Code suggestions were accepted only after the tests passed and the outputs agreed with hand calculations or analytical solutions. Numerical claims in this report were checked against the stored results files. I questioned the assistant's work in several places: I challenged an early validation scheme because the same data appeared to train and test the model, which led to the second validation method; and I required that the repository code be written by me and not by the assistant.

Errors and unhelpful suggestions. The assistant made several errors that I had to identify. An early instruction used a virtual-environment path that did not exist in my project; advice to push a new branch omitted the upstream option; early file edits were made directly in my repository before I asked that they stop; a cost-ordering test did not detect a deliberate fault until I asked for it to be tightened; and a test file I was given failed on first run because of an import path that did not match my project layout.

Effect on understanding. The assistant shortened the time spent on set-up and let me spend more time on interpretation, but it also created a risk of accepting plausible explanations without understanding them. I am responsible for all code, results, claims and references submitted.

REFERENCES

AEMO (2026) *Aggregated price and demand data*, Victoria (VIC1). Australian Energy Market Operator, accessed 6 October 2026. aemo.com.au.

Bergmeir, C. and Benítez, J.M. (2012) On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191, 192–213.

Huangfu, Q. and Hall, J.A.J. (2018) Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119–142.

Hyndman, R.J. and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice*, 3rd edn. OTexts, Melbourne.

López de Prado, M. (2018) *Advances in Financial Machine Learning*. Wiley, Hoboken (purged validation).

Pedregosa, F. et al. (2011) Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Smola, A.J. and Schölkopf, B. (2004) A tutorial on support vector regression. *Statistics and Computing*, 14(3), 199–222.

Virtanen, P. et al. (2020) SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

Zippenfenig, P. (2023) Open-Meteo.com Weather API. Zenodo, doi:10.5281/zenodo.7970649.

Ambrose, M. (CSIRO) (2026) rbee household electricity use extract by hour and state. Personal communication; data supplied to the author.

APPENDIX A. SENSITIVITY RESULTS

Table A1. Annual bill ($) by controller for each sensitivity case (359 days).

Case	A	B	C	D	C saves vs B	Error cost (C–D)
Base case	1,179.84	643.46	590.75	546.13	52.71	44.62
Battery 5 kWh	1,179.84	820.15	805.04	769.56	15.10	35.48
Battery 15 kWh	1,179.84	628.66	510.16	448.96	118.50	61.20
Battery 20 kWh	1,179.84	623.05	463.37	391.03	159.68	72.33
Round-trip efficiency 80%	1,179.84	668.84	633.25	590.21	35.59	43.04
Round-trip efficiency 85%	1,179.84	655.37	611.87	567.94	43.50	43.93
Round-trip efficiency 95%	1,179.84	632.29	569.65	524.32	62.63	45.34
Battery power 2.5 kW	1,179.84	643.51	591.98	561.66	51.53	30.32
Battery power 7.5 kW	1,179.84	643.46	590.90	535.20	52.56	55.70
Feed-in fixed 5 c/kWh	1,000.76	548.50	530.93	517.80	17.57	13.13
Network charge 10 c/kWh	992.67	587.42	532.23	480.56	55.19	51.67
Network charge 20 c/kWh	1,367.00	699.50	649.94	607.30	49.56	42.65
Forecast horizon 96 h	1,179.84	643.46	590.26	546.13	53.20	44.12
Forecast horizon 120 h	1,179.84	643.46	590.52	546.13	52.94	44.38

APPENDIX B. REPOSITORY GUIDE AND REPRODUCTION

The repository (github.com/lithdhar/MMA3001-Project) holds the source modules in *src/*, tests in *tests/*, notebooks 01 to 17 in the order of the analysis, original and processed data in *data/*, results and figures in *results/*, HTML documentation generated with pdoc in *docs/*, the online advisor in *app/* and a README with set-up instructions. To reproduce the results, install the packages in *requirements.txt* (Python 3.10), run the notebooks in numerical order, and run *pytest* from the project root. Notebooks 04 and 14 are the longest, at roughly one minute each on a laptop.